

Problem Set Solutions: Seminar 03

Sorting (2)
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: Inversions, selection, greedy orderings, exchange arguments
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Sorting is not only a procedure for putting numbers in order. In this seminar, sorting appears as a structural tool: insertion sort measures disorder through inversions; selection avoids sorting the whole array; greedy orderings are proved by adjacent exchanges; and stable partitions reveal exactly which local inversions must be corrected. The recurring theme is to identify the local comparison that determines the global optimum.

Contents

Problem 1. Insertion sort and inversions	2
Problem 2. The recursive supermedian is not a balanced pivot	2
Problem 3. The first k order statistics in $O(n \log k)$	3
Problem 4. Choosing k pairs to maximize a mixed sum	4
Problem 5. Optimal play on pairs (a_i, b_i)	5
Problem 6. Choosing n boxes with at least half of each fruit	5
Problem 7. Sorting by mex replacements in at most $2n$ operations	6
Problem 8. Stable threshold partitions of a permutation	7
Problem 9. Lullabies and one common sleeping minute	8
Problem 10. Ordering affine budget transformations	9
Problem 11. Scheduling orders with linear waiting penalties	10

Conventions. Arrays are indexed from 1 unless stated otherwise. In the mex problem we explicitly switch to zero-based indices. Ties in sorting keys may be broken arbitrarily unless the proof states otherwise. All logarithms may have any fixed base greater than 1.

Problem 1. Insertion sort and inversions

Consider insertion sort: the algorithm scans the array from left to right, keeps the current prefix sorted, and pushes the next element left until the prefix is sorted again. Estimate its running time in O and Ω notation. Prove the precise dependence on the number of inversions in the initial array.

Solution.

Idea. Insertion sort is adaptive: it is fast on arrays that are already close to sorted. The correct measure of disorder is the number of inversions

$$\text{Inv}(a) = \#\{(i, j) : 1 \leq i < j \leq n, a_i > a_j\}.$$

Algorithm. Run the standard adjacent-swap implementation of insertion sort. In the inner loop, while the current element is smaller than its predecessor, swap these two adjacent elements.

Correctness. Every adjacent swap performed by insertion sort swaps two adjacent elements a_{t-1}, a_t with $a_{t-1} > a_t$. Before the swap this adjacent pair is an inversion; after the swap it is not. Since the elements are adjacent, the relative order of every other pair of elements is unchanged. Hence each such swap decreases the total number of inversions by exactly 1.

The algorithm terminates with a sorted array, and a sorted array has zero inversions. Therefore the number of adjacent swaps performed by insertion sort is exactly $\text{Inv}(a)$.

The outer scan and final failed comparisons contribute linear overhead. Thus the precise asymptotic form is

$$T(a) = \Theta(n + \text{Inv}(a)).$$

In particular, since $0 \leq \text{Inv}(a) \leq \binom{n}{2}$, insertion sort has best-case time $\Theta(n)$ and worst-case time $\Theta(n^2)$.

Running time. Globally, insertion sort is $O(n^2)$ and $\Omega(n)$. More precisely, on input a it runs in $\Theta(n + \text{Inv}(a))$ time and uses $O(1)$ extra memory.

Problem 2. The recursive supermedian is not a balanced pivot

For $n = 5$, call the median of the array its supermedian. For $n = 5^k$, split the array into consecutive groups of five, take the median of each group, and define the supermedian recursively as the supermedian of these medians. Prove that there is no universal constant $C \in (0, 1)$ such that the supermedian is always an order statistic with rank in $[Cn, (1 - C)n]$.

Solution.

Idea. A median of five gives a local guarantee, but recursively taking medians of medians in this specific way does not preserve a constant-fraction rank guarantee. We construct inputs where the final supermedian has rank only 3^t among 5^t elements.

Construction. Let $n = 5^t$. We show by induction that there exists an array of length 5^t whose supermedian has rank exactly 3^t , where rank 1 means the minimum element.

For $t = 1$, the supermedian is the ordinary median of five elements, so its minimum possible rank is $3 = 3^1$.

Assume the claim for $t - 1$. We prescribe the 5^{t-1} group medians $b_1, \dots, b_{5^{t-1}}$ so that their recursive supermedian is an element b_* of rank 3^{t-1} among these medians.

Now realize each median b_i by a group of five original elements. If $b_i < b_*$, choose the group so that exactly three of its five elements are below b_* : two elements below b_i , then b_i , and two elements far above b_* . For the group containing b_* , choose two elements below b_* , then b_* , then two elements above b_* . If $b_i > b_*$, choose the group so that no element is below b_* except what is forced by making b_i the median; in particular, all five elements may be chosen above b_* while preserving the prescribed median order.

Correctness. The first-level medians are exactly the prescribed values $b_1, \dots, b_{5^{t-1}}$, so the recursive supermedian of the whole array is still b_* . Among the prescribed medians, exactly 3^{t-1} medians are at most b_* . Each corresponding group contributes exactly three original elements at most b_* . All groups with median greater than b_* can be chosen to contribute no elements below b_* . Therefore the rank of b_* in the original array is exactly

$$3 \cdot 3^{t-1} = 3^t.$$

Hence

$$\frac{3^t}{5^t} = \left(\frac{3}{5}\right)^t \rightarrow 0.$$

For any fixed $C > 0$, choosing t large enough gives $3^t < C5^t$, so the supermedian may have rank smaller than Cn . By reversing all values, we obtain a symmetric construction where the rank is larger than $(1 - C)n$. Thus no such universal constant C exists.

Running time. This is a counterexample construction rather than an algorithm. Its algorithmic consequence is that this pivot rule can produce highly unbalanced quicksort partitions.

Problem 3. The first k order statistics in $O(n \log k)$

Given an array of length n , find the first k order statistics, i.e. the k smallest elements with multiplicities, in $O(n \log k)$ time.

Solution.

Idea. If only the smallest k elements are needed, we should never fully sort all n elements. Maintain only the current best k candidates.

Algorithm. Maintain a max-heap H of size at most k .

1. Insert the first k elements into H .
2. For each remaining element x , compare it with $\max(H)$.
 - If $x \geq \max(H)$, ignore x .
 - If $x < \max(H)$, delete $\max(H)$ and insert x .
3. At the end, H contains the k smallest elements. If they must be printed in sorted order, repeatedly extract the maximum and reverse the resulting list.

Correctness. We prove the invariant that after processing the first i elements, the heap contains exactly the $\min(k, i)$ smallest among them.

For $i \leq k$, every processed element is inserted, so the invariant is immediate. Suppose $i > k$ and the invariant holds before reading $x = a_i$. If $x \geq \max(H)$, then the heap already contains k elements no larger than x , so x cannot belong to the k smallest processed elements. Ignoring it preserves the invariant. If $x < \max(H)$, then the current heap maximum is worse than x and cannot remain among the k smallest after x appears. Replacing that maximum by x restores exactly the k smallest elements. Therefore the invariant holds after every step.

At the end of the scan, H contains exactly the first k order statistics.

Running time. Each heap operation costs $O(\log k)$, and each input element causes at most one insertion and one deletion. The total time is $O(n \log k)$. The heap uses $O(k)$ memory.

Problem 4. Choosing k pairs to maximize a mixed sum

Given n pairs $(a_1, b_1), \dots, (a_n, b_n)$ and an integer k , choose a set $I \subseteq \{1, \dots, n\}$ of size k maximizing

$$\sum_{i \in I} a_i + \sum_{i \notin I} b_i.$$

Find the maximum value in $O(n \log n)$ time.

Solution.

Idea. The expression becomes one-dimensional after subtracting the contribution obtained by choosing all b_i 's.

Algorithm. Rewrite the objective as

$$\sum_{i=1}^n b_i + \sum_{i \in I} (a_i - b_i).$$

The first term is constant. Define $d_i = a_i - b_i$. Sort the values d_i in nonincreasing order and take the largest k of them. The maximum value is

$$\sum_{i=1}^n b_i + \sum_{j=1}^k d_{(j)}$$

where $d_{(1)} \geq d_{(2)} \geq \dots \geq d_{(n)}$.

Correctness. Suppose an optimal set I contains an index p but omits an index q with $d_q > d_p$. Replacing p by q changes the objective by

$$d_q - d_p > 0,$$

which contradicts optimality. Hence no optimal set can take a smaller marginal gain while omitting a larger one. Therefore an optimal set consists of exactly the k largest values $d_i = a_i - b_i$.

Running time. Computing all d_i and $\sum b_i$ takes $O(n)$ time. Sorting takes $O(n \log n)$ time. The memory usage is $O(n)$ if the differences are stored.

Problem 5. Optimal play on pairs (a_i, b_i)

There are n pairs (a_i, b_i) of positive integers. Two players move alternately. On each move, the current player chooses one previously unchosen pair. If the first player chooses pair i , he gains a_i ; if the second player chooses pair i , she gains b_i . Let S_1, S_2 be the final sums. The first player maximizes $S_1 - S_2$, and the second minimizes it. Find the value of $S_1 - S_2$ under optimal play in $O(n \log n)$ time.

Solution.

Idea. The importance of item i is not a_i alone or b_i alone. If the first player takes it, the value increases by a_i ; if the second player takes it, the value decreases by b_i . Thus the total swing is $a_i + b_i$.

Algorithm. Sort the pairs by nonincreasing

$$w_i = a_i + b_i.$$

Then the players take pairs in this order. The first player receives a_i on odd positions of the sorted order, and the second player receives b_i on even positions. Therefore the game value is

$$\boxed{\sum_{r \text{ odd}} a_{\pi_r} - \sum_{r \text{ even}} b_{\pi_r}},$$

where $w_{\pi_1} \geq w_{\pi_2} \geq \dots \geq w_{\pi_n}$.

Correctness. Consider two remaining items x and y and two consecutive moves.

If it is the first player's move, taking x now and leaving y to the second player contributes $a_x - b_y$ to $S_1 - S_2$. Taking y now and leaving x contributes $a_y - b_x$. The first player prefers x before y exactly when

$$a_x - b_y \geq a_y - b_x \iff a_x + b_x \geq a_y + b_y.$$

If it is the second player's move, taking x now and leaving y to the first player contributes $-b_x + a_y$. The second player wants to minimize the value, so she prefers x before y exactly when

$$-b_x + a_y \leq -b_y + a_x \iff a_x + b_x \geq a_y + b_y.$$

Thus on every move both players choose a remaining item with maximum $a_i + b_i$. The sorted greedy order is forced up to ties, and the displayed alternating sum is the value of optimal play.

Running time. Sorting by $a_i + b_i$ costs $O(n \log n)$ time. Computing the alternating sum costs $O(n)$ time.

Problem 6. Choosing n boxes with at least half of each fruit

There are $2n - 1$ boxes containing apples and oranges. Prove that one can always choose n boxes containing at least half of all apples and at least half of all oranges.

Solution.

Idea. Sort by one fruit. Then construct two overlapping n -element candidate sets that are both good for that fruit. Since their apple sums add to at least the total number of apples, one of them is also good for apples.

Proof. Sort the boxes by the number of oranges:

$$o_1 \leq o_2 \leq \dots \leq o_{2n-1}.$$

Define two n -element sets

$$S_{\text{odd}} = \{1, 3, 5, \dots, 2n-1\},$$

$$S_{\text{even}} = \{2, 4, 6, \dots, 2n-2, 2n-1\}.$$

Both sets contain at least half of all oranges. Indeed, for S_{odd} we compare o_{2j} with o_{2j+1} for $1 \leq j \leq n-1$; since $o_{2j} \leq o_{2j+1}$, the odd-indexed set has orange sum at least the orange sum of its complement. For S_{even} , compare o_{2j-1} with o_{2j} for $1 \leq j \leq n-1$, and note that o_{2n-1} is included in S_{even} . Hence this set also has at least half of all oranges.

Now look at apples. The two sets together contain every box once, and box $2n-1$ twice. Therefore

$$A(S_{\text{odd}}) + A(S_{\text{even}}) = A_{\text{total}} + A_{2n-1} \geq A_{\text{total}}.$$

So at least one of these two sets has apple sum at least $A_{\text{total}}/2$. Since both candidate sets already have at least half of all oranges, that candidate set satisfies both requirements.

Running time. The proof is existential. It also gives an $O(n \log n)$ algorithm: sort by oranges, form the two candidate sets, and choose the one with larger apple sum.

Problem 7. Sorting by mex replacements in at most $2n$ operations

For an array $a_1, \dots, a_n$, let $\text{mex}(a)$ be the smallest nonnegative integer absent from the array. In one operation we may replace any one array element by the current mex. Show how to make the array nondecreasing in at most $2n$ operations, with $O(n)$ implementation time.

Solution.

Idea. It is enough to transform the array into

$$[0, 1, 2, \dots, n-1],$$

which is nondecreasing. Use zero-based indices in this problem.

Algorithm. Maintain the current mex m of the array.

1. If $m < n$, replace a_m by m .
2. If $m = n$ and the array is not yet equal to $[0, 1, \dots, n-1]$, choose an index i with $a_i \neq i$ and replace a_i by n .
3. Update the mex and repeat.

Correctness. Suppose first that $m < n$. Since m is absent from the array, replacing a_m by m makes position m correct. It does not destroy any already correct position: if $a_j = j$, then the value j is present, so $j \neq m$.

Now suppose $m = n$. Then every value $0, 1, \dots, n-1$ is present, so the array is a permutation of these values. If it is not already sorted, pick an incorrect index i with $a_i \neq i$ and write n there. This makes the old value a_i absent, so the next mex is some value below n . The following operation of the first type writes that missing value into its own position, fixing at least one new position.

Thus every operation of the first type fixes one new position, and every operation of the second type is used only to create a missing value below n and is followed by an operation that fixes a new position. No fixed position is ever destroyed. Therefore at most $2n$ operations are needed.

Running time. The number of operations is at most $2n$. With a frequency array for values $0, \dots, n$ and a bounded-universe missing-value structure for the current mex, updates are constant-time amortized on the standard word-RAM model, so the total time is $O(n)$. A balanced-tree implementation gives the same construction in $O(n \log n)$ time.

Problem 8. Stable threshold partitions of a permutation

Given a permutation $p_1, \dots, p_n$, one operation chooses a threshold x , writes first all numbers $< x$ in their current relative order, then all numbers $\geq x$ in their current relative order, and replaces the permutation by the result. Find the minimum number of operations needed to sort the permutation.

Solution.

Idea. The operation is stable on both sides of the threshold. Therefore it can change the relative order of consecutive values i and $i+1$ only when the threshold is exactly $i+1$.

Algorithm. Let $pos[v]$ be the position of value v in the initial permutation. Count

$$D = \{i \in \{1, \dots, n-1\} : pos[i] > pos[i+1]\}.$$

The answer is

$$\boxed{|D|}.$$

Correctness. For the lower bound, suppose $pos[i] > pos[i+1]$. Then the consecutive values i and $i+1$ are in the wrong relative order. A stable threshold operation separates them only if

$$i < x \leq i+1,$$

which means $x = i+1$. For every other threshold, both values lie on the same side of the partition, so their relative order is preserved. Hence every such descent forces at least one operation with threshold $i+1$. Therefore at least $|D|$ operations are necessary.

For the upper bound, split the values into maximal consecutive blocks such that inside each block the values already occur in increasing relative order in the initial permutation. The boundaries between these blocks are exactly the indices in D . Apply the operation with thresholds $i+1$ for all $i \in D$, in increasing order of i . After the first such operation, the first block moves before all later values while preserving its internal order. Inductively, after processing the first r boundaries, the first $r+1$ blocks are placed before all remaining blocks in correct block order, and the internal order

inside each block is preserved. After all boundaries are processed, the blocks appear in increasing value order and each block is internally increasing, so the whole permutation is sorted.

The lower and upper bounds match, so the minimum number of operations is $|D|$.

Running time. Compute the inverse permutation pos in $O(n)$ time and count the descents $pos[i] > pos[i+1]$ in another $O(n)$ time. The memory usage is $O(n)$.

Problem 9. Lullabies and one common sleeping minute

There are n children. To put child i to sleep, the mother must sing for a_i minutes; after that, the child sleeps for b_i consecutive minutes. Determine whether the children can be handled so that there is at least one free minute during which all children are asleep. Give an $O(n \log n)$ algorithm.

Solution.

Idea. After all singing is completed, the last completion time is the only possible beginning of a common free interval in a no-idle schedule. A child sung too early may wake before then, so children with larger $a_i + b_i$ should be scheduled earlier.

Algorithm. Let

$$T = \sum_{i=1}^n a_i.$$

Sort children by nonincreasing $a_i + b_i$. In this order, compute each completion time C_i . Answer YES if and only if

$$C_i + b_i \geq T + 1$$

for every child i .

If the problem is interpreted as asking for a common instant rather than a full minute interval, replace $T + 1$ by T everywhere; the ordering argument is identical.

Correctness. First, idle time is never useful. If a feasible schedule contains idle time, remove an idle interval. Jobs after the idle interval and the final finishing time shift left by the same amount, while jobs before the idle interval become only safer relative to the final finishing time. Thus feasibility is preserved. Hence it suffices to consider no-idle schedules, whose final singing time is T .

In a no-idle schedule, child i is asleep for the whole free minute $[T, T + 1)$ exactly when

$$C_i + b_i \geq T + 1.$$

If child i starts at time S_i , this is equivalent to

$$S_i \geq T + 1 - a_i - b_i.$$

Thus each child has a lower bound on its start time

$$r_i = T + 1 - a_i - b_i.$$

Since $T + 1$ is common to all children, sorting by nondecreasing r_i is the same as sorting by nonincreasing $a_i + b_i$.

It remains to prove that this sorted order is sufficient whenever any feasible order exists. Consider two adjacent children i, j in a feasible schedule after a prefix of length P , and suppose they are inverted with respect to the release lower bounds: $r_i > r_j$, but i is scheduled before j . In the original order, feasibility gives $P \geq r_i$ and $P + a_i \geq r_j$. If we swap them, child j starts at P , and since $r_j < r_i \leq P$, it is feasible. Child i starts at $P + a_j$, and since $P \geq r_i$, it is also feasible. Thus swapping adjacent inversions preserves feasibility. Repeating this operation transforms any feasible schedule into the sorted order. Therefore checking the sorted order is necessary and sufficient.

Running time. Sorting costs $O(n \log n)$. The feasibility scan costs $O(n)$.

Problem 10. Ordering affine budget transformations

Initially the city has annual income k . It must build n buildings in some order. If before building i the income is x , then after building it the income becomes $a_i x - b_i$, where $a_i, b_i \geq 0$ are not necessarily integers. Determine an order maximizing the final income in $O(n \log n)$ time.

Solution.

Idea. Each building applies an affine map $f_i(x) = a_i x - b_i$. The order matters because affine maps do not commute. The correct global order is obtained by comparing two adjacent buildings.

Algorithm. For two buildings i and j , placing i before j is at least as good as placing j before i exactly when

$$b_i(a_j - 1) \leq b_j(a_i - 1).$$

Sort according to this comparator. Equivalently, use the following explicit order:

1. all buildings with $a_i > 1$, sorted by increasing $\frac{b_i}{a_i - 1}$;
2. all buildings with $a_i = 1$, in arbitrary order;
3. all buildings with $a_i < 1$, sorted by increasing $\frac{b_i}{a_i - 1}$.

Correctness. Compare two adjacent buildings i and j after some current income x . If i is built before j , the resulting income is

$$f_j(f_i(x)) = a_j a_i x - a_j b_i - b_j.$$

If j is built before i , the resulting income is

$$f_i(f_j(x)) = a_i a_j x - a_i b_j - b_i.$$

The coefficient of x is the same in both expressions. Therefore i before j is at least as good as j before i if and only if

$$-a_j b_i - b_j \geq -a_i b_j - b_i,$$

which is equivalent to

$$b_i(a_j - 1) \leq b_j(a_i - 1).$$

This condition is independent of the current income x and of the rest of the order. Hence any adjacent pair violating the condition can be swapped without decreasing the final income. Repeatedly

swapping such inversions leads to the sorted order above, and every swap is nondecreasing for the objective. Therefore the sorted order is optimal.

The three-part explicit rule is just the same comparator written without division by zero: buildings with $a > 1$ precede those with $a = 1$, which precede those with $a < 1$; inside the groups $a > 1$ and $a < 1$, the comparator becomes sorting by increasing $b/(a - 1)$.

Running time. Sorting with the comparator takes $O(n \log n)$ time. Computing the final income after sorting takes $O(n)$ time.

Problem 11. Scheduling orders with linear waiting penalties

A factory receives n orders. Order i takes t_i days to complete, and each day of delay costs penalty f_i . Only one order can be processed at a time. Determine an order minimizing the total penalty.

Solution.

Idea. This is Smith's rule. If job i waits one extra day, the objective increases by f_i ; if it is processed now, it occupies t_i days. The correct priority is penalty per processing day.

Algorithm. Sort the orders by nonincreasing ratio

$$\frac{f_i}{t_i}.$$

Equivalently, compare two jobs i, j by cross-multiplication: place i before j if

$$f_i t_j \geq f_j t_i.$$

Correctness. Let C_i be the completion time of order i . The total penalty is

$$\sum_i f_i C_i.$$

Consider two adjacent orders i, j after a prefix of total length P . If we process i then j , their contribution is

$$f_i(P + t_i) + f_j(P + t_i + t_j).$$

If we process j then i , their contribution is

$$f_j(P + t_j) + f_i(P + t_j + t_i).$$

The first order is no worse than the second exactly when

$$f_j t_i \leq f_i t_j \iff \frac{f_i}{t_i} \geq \frac{f_j}{t_j}.$$

Therefore every adjacent inversion by the ratio f_i/t_i can be swapped without increasing the objective. Repeatedly eliminating such inversions yields an order sorted by nonincreasing f_i/t_i , and the total penalty never increases during this process. Hence Smith's rule is optimal.

Running time. Sorting costs $O(n \log n)$ time. After sorting, computing completion times and the total penalty costs $O(n)$ time.